

Internship report

Djamila Mohamed

Supervisor: Adel Djoudi, Formal Methods Expert

Referent Professor: Ahmed Bouajjani

May 28, 2024



Contents

1	The company	2
1.1	DIS-IBS branch	2
1.2	Organization chart	2
2	Topic of the internship	2
3	Study of a solution for code editor users	3
3.1	Background	3
3.2	Language Servers	4
3.2.1	Enhanced Performance	4
3.2.2	Language Agnosticism	4
3.2.3	Centralized Maintenance	4
3.2.4	Extensibility	5
3.2.5	Examples of Language Servers	5
4	Frama-C plugin development	5
5	Future work	5

1 The company

1.1 DIS-IBS branch

1.2 Organization chart

text

2 Topic of the internship

Java card, EAL certif etc.

My role as an intern at Thales DIS is to develop a code assistance tool for Frama-C developers who use ACSL annotations in their C source code. ACSL (ANSI/ISO C Specification Language) is a language used to specify the behaviour of programs written using the C language, it is used to ensure the correctness of functions

and verify their behaviour corresponds to their specification.

3 Study of a solution for code editor users

3.1 Background

Developers frequently encounter difficulties when selecting an appropriate development environment for their software projects. Many text editors consume excessive storage capacity and tend to specialize their features for only a handful of popular programming languages. For instance, consider JetBrains' IntelliJ IDEA (Community edition), which offers comprehensive Java tooling and libraries but requires payment for JavaScript projects and other Java framework support. This limitation applies to other well-known text editors as well [2].

Developers desire a versatile and lightweight Integrated Development Environment (IDE) that includes essential code assistance tools. They seek a tool that encompasses the functionality found in standard editors, such as real-time code completion, efficient navigation to method or variable definitions, and comprehensive information display when hovering over a symbol. Furthermore, it is crucial that these features are promptly accessible to users. To address these shortcomings, a recent solution has emerged involving client-server communication between the editor and a language server [3].

3.2 Language Servers

The utilization of Language Servers proves advantageous in our present situation. The Language Server Protocol (LSP) establishes a standardized means of communication between editors/IDEs and language servers. This protocol defines a set of requisite methods and data structures that language servers must incorporate, providing integration capabilities with a broad array of development tools. By employing a language server, editors and IDEs can delegate language-specific processing tasks to the server. Consequently, the server delivers code comprehension and coding features promptly and efficiently to the client. This approach presents several benefits:

3.2.1 Enhanced Performance

Language servers can offload resource-intensive language analysis tasks, such as code completion, symbol lookup, and hover information, to a dedicated server. In doing so, editors and IDEs are relieved of this burden, resulting in swifter response times and a more responsive development experience for users.

3.2.2 Language Agnosticism

Language servers are designed to be language-agnostic, allowing a single language server to support multiple programming languages. This empowers developers to utilize their preferred editor or IDE across a wide range of languages, eliminating the need for constant switching between different development environments.

3.2.3 Centralized Maintenance

Language servers are independently maintained and updated, separate from the editor or IDE. This makes it easier to ensure the latest language-specific features are up-to-date. Developers can take advantage of recent improvements and bug fixes without the obligation to update their entire development environment.

3.2.4 Extensibility

Language servers can be extended to accommodate additional features or tailored to suit the specific requirements of a development team or project. This allows for a more personalized development experience aligned with the team's workflows and preferences. For further information regarding LSP, refer to .

3.2.5 Examples of Language Servers

A notable instance of a recent language server is the Nickel Language Server developed by Yannik Sander under the Tweag company (which also developed the Nickel configuration language).

4 Frama-C plugin development

5 Future work

6 Bibliography